

UD7 – Formularios y validación de datos

1. Introducción

Los formularios son el medio principal de interacción entre el usuario y una aplicación web. A través de ellos, los usuarios introducen datos que pueden ser enviados, almacenados o procesados. Sin embargo, esta comunicación entre personas y software implica un riesgo: los usuarios pueden cometer errores, o incluso intentar manipular la información que envían.

Por eso la validación y la sanitización de los datos son aspectos críticos en cualquier desarrollo web. Validar significa comprobar que los datos cumplen con unas reglas (por ejemplo, “el campo correo debe tener una @”). Sanitizar significa limpiar los datos para eliminar cualquier contenido potencialmente dañino (por ejemplo, un `<script>` oculto en un campo de texto).

JavaScript nos permite hacer todo esto en el lado del cliente, antes de que los datos lleguen al servidor. El objetivo no es solo la seguridad, sino también la usabilidad: evitar que el usuario pierda tiempo enviando formularios incompletos o con errores.

Comparativa con otros lenguajes

Lenguaje	Dónde valida	Observación
JavaScript	Cliente (navegador)	Interacción rápida, pero vulnerable si el usuario desactiva JS.
PHP / Python / Java	Servidor	Mayor seguridad, pero más lento (requiere reenvío del formulario).
Full Stack (cliente + servidor)	Ambos lados	La opción profesional: experiencia fluida + validación segura.

2. Conceptos clave y fundamentos

2.1. Tipos de validación

Tipo	Dónde ocurre	Ejemplo
HTML5 nativa	En el propio navegador	Atributos como <code>required</code> , <code>pattern</code> , <code>type="email"</code>

JavaScript manual	Mediante scripts personalizados	Comprobaciones dinámicas y mensajes personalizados
Servidor	Tras enviar el formulario	Validación en PHP, Python, Node.js, etc.

2.2. Validación en HTML5

HTML5 incorpora muchos mecanismos automáticos de validación:

```
<form>

  <input type="email" required placeholder="Correo electrónico">

  <input type="number" min="0" max="120" required placeholder="Edad">

  <input type="text" pattern="[A-Za-z\s]+" placeholder="Solo letras">

  <button type="submit">Enviar</button>

</form>
```

Estos atributos ya impiden el envío si no se cumple la condición, pero presentan dos limitaciones:

1. Los mensajes son genéricos y dependen del navegador.
2. No permiten validaciones complejas ni personalizadas (como DNI o contraseñas fuertes).

Por eso, en el desarrollo profesional, se complementan con validación JavaScript.

2.3. Interceptar el envío: `submit` y `preventDefault()`

Cuando un formulario se envía, se ejecuta el evento `submit`. Si no lo controlamos, la página se recarga. Para evitarlo:

```
form.addEventListener("submit", (e) => {

  e.preventDefault();

  validarFormulario();

});
```

Nota:

`preventDefault()` evita la acción por defecto (enviar y recargar).

El evento se dispara en el botón `"submit"`, no en un clic cualquiera.

2.4. Expresiones regulares (RegEx)

Las **RegEx** permiten definir patrones de texto que deben cumplir los campos del formulario.

Ejemplo básico

```
let nombre = "María";

let patron = /^[A-Za-zÁÉÍÓÚáéíóúÑñ\s]{3,}$/;

if (patron.test(nombre)) {
    console.log("Nombre válido");
} else {
    console.log("Nombre no válido");
}
```

Explicación:

- `^` → Inicio de cadena.
- `[A-Za-zÁÉÍÓÚáéíóúÑñ\s]` → Letras y espacios (permite tildes y ñ).
- `{3,}` → Mínimo tres caracteres.
- `$` → Fin de cadena.

2.5. Cuantificadores y metacaracteres principales

Símbolo	Significado	Ejemplo	Coincide con
<code>^</code>	Inicio	<code>^Hola</code>	"Hola mundo"
<code>\$</code>	Fin	<code>mundo\$</code>	"Hola mundo"
<code>.</code>	Cualquier carácter	<code>H.la</code>	"Hola"
<code>+</code>	Uno o más	<code>a+</code>	"aaa"
<code>*</code>	Cero o más	<code>ba*</code>	"b", "ba", "baa"
<code>?</code>	Cero o uno	<code>colou?r</code>	"color", "colour"
<code>{n,m}</code>	Entre n y m repeticiones	<code>\d{8}</code>	8 dígitos

2.6. Ejemplos de validación avanzada

Campo	Expresión regular	Comentario
DNI español	<code>/^\d{8}[A-Z]\$/</code>	Letra mayúscula final
Teléfono español	<code>/^[67]\d{8}\$/</code>	9 dígitos, empieza por 6 o 7
Correo electrónico	<code>/^[\w._%+-]+@[\w.-]+\.[a-zA-Z]{2,}\$/</code>	Simplificada
Contraseña fuerte	<code>/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@\$!%*?&]).{8,}\$/</code>	Incluye mayúsculas, minúsculas, número y símbolo
Fecha dd/mm/yyyy	<code>^\d{0}[1-9]</code>	<code>[12][0-9]</code>

2.7. Gestión de errores con **try**, **catch** y **throw**

La validación puede generar errores controlados:

```
try {  
    let edad = parseInt(prompt("Introduce tu edad"));  
    if (isNaN(edad)) throw new Error("No es un número válido");  
    if (edad < 18) throw new Error("Debes ser mayor de edad");  
    alert("Acceso permitido");  
} catch (error) {  
    alert("Error: " + error.message);  
}  
finally {  
    console.log("Validación completada");  
}
```

Nota:

throw lanza el error; **catch** lo gestiona.

En validaciones complejas se usa para detener el flujo cuando un campo falla.

3. Ejemplo modular de formulario completo

```
<form id="registro">

  <label>Nombre</label>

  <input type="text" id="nombre">

  <label>Correo</label>

  <input type="email" id="correo">

  <label>Contraseña</label>

  <input type="password" id="clave">

  <button type="submit">Registrar</button>

  <p id="errores"></p>

</form>
```

```
const form = document.getElementById("registro");
```

```
const errores = document.getElementById("errores");
```

```
form.addEventListener("submit", e => {
```

```
  e.preventDefault();
```

```
  errores.innerHTML = "";
```

```
  let mensajes = [];
```

```
  if (!/^[A-Za-z\s]{3,}$/.test(nombre.value)) {
```

```
    mensajes.push("El nombre debe tener al menos 3 letras.");
```

```
  }
```

```
  if (!/^[\\w._%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}$/.test(correo.value)) {
```

```
    mensajes.push("Correo no válido.");  
}  
  
if (!/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d){8,}$/.test(clave.value)) {  
    mensajes.push("La contraseña debe tener 8 caracteres, mayúscula y  
número.");  
}  
  
if (mensajes.length > 0) {  
    errores.innerHTML = mensajes.join("<br>");  
    errores.style.color = "red";  
} else {  
    alert("Registro completado con éxito");  
}  
});
```

Análisis:

- Cada validación es independiente.
- Se acumulan los errores en un array (`mensajes`).
- El formulario solo se envía si el array está vacío.
- Usa `preventDefault()` para evitar recarga.

4. Errores comunes y buenas prácticas

 Error	 Solución
Usar <code>return false</code> en lugar de <code>preventDefault()</code>	<code>preventDefault()</code> es más fiable y moderno
Validar antes de limpiar espacios	Usa <code>.trim()</code> antes de testear
Usar <code>innerHTML</code> para mensajes de usuario	Sustituir por <code>.textContent</code> para evitar XSS

Mezclar validación visual con lógica	Separar funciones: <code>validarCampo()</code> y <code>mostrarError()</code>
Confiar solo en validación cliente	Repetir validación en servidor

✓ Buenas prácticas

- Usa `const` y `let` (nunca `var`).
- Crea funciones reutilizables (`validarCorreo`, `validarDNI`, etc.).
- Encapsula el código dentro de `window.onload`.
- Añade feedback visual (colores, iconos, bordes dinámicos).

5. Curiosidades y ampliación

Los frameworks modernos (React, Vue, Angular) implementan validación mediante hooks o directivas que actúan igual que las RegEx, pero integradas en el ciclo de componentes.

En entornos profesionales se combinan RegEx con validación asíncrona: por ejemplo, comprobar en segundo plano si un usuario o correo ya existen (esto conecta con la UD7 – JSON y AJAX).

Librerías populares para validar formularios: Validator.js, Formik, Yup, JustValidate.

6. Ejercicios prácticos y autoevaluación

1. Diseña un formulario con campos: Nombre, Correo, Contraseña, Repite Contraseña.
 - Implementa validación en tiempo real.
 - Muestra los errores en una lista bajo el formulario.
2. Valida un **DNI** comprobando que la letra sea correcta (usa el algoritmo de módulo 23).
3. Crea un **formulario de contacto** que solo muestre el mensaje “Enviado con éxito” si todos los campos son válidos.
4. Amplía el formulario para guardar los datos localmente con `localStorage` (opcional, preparación para UD7).
5. **Reto final:** divide tu código en funciones (`validarCampo`, `validarFormulario`, `mostrarErrores`) y documenta cada una con comentarios.

7. Cuadro de referencia rápida

Categoría	Elemento	Descripción / Ejemplo
Eventos	<code>submit</code> , <code>input</code> , <code>blur</code> , <code>change</code>	Controlan la interacción del usuario

Métodos RegEx	<code>test()</code> , <code>exec()</code> , <code>match()</code> , <code>replace()</code> , <code>split()</code>	Operaciones sobre cadenas
Validación HTML5	<code>required</code> , <code>pattern</code> , <code>min</code> , <code>max</code> , <code>maxlength</code> , <code>type</code>	Atributos nativos
Gestión de errores	<code>try</code> , <code>catch</code> , <code>throw</code> , <code>finally</code>	Control de excepciones
Funciones útiles	<code>.trim()</code> , <code>.toLowerCase()</code> , <code>.toUpperCase()</code>	Limpieza y normalización de datos
Seguridad	<code>.textContent</code> vs <code>.innerHTML</code>	Evita inyecciones de código (XSS)